

Ejercicios 23-Oct-2025

EJERCICIOS DE MODELOS ANIDADADOS + API + TRY/CATCH

(Dart puro, sin Flutter)

Ejercicio 1 — Comprender la estructura del JSON anidado

Objetivo: familiarizarse con el formato que devuelve la API.

Instrucciones:

1. Haz una petición simple con `http.get(Uri.parse('https://restcountries.com/v2/name/peru'))`.
 2. Imprime la estructura del JSON completa.
 3. Identifica visualmente los niveles de anidación:
 - `Country` → `name`
 - `Country` → `currencies`
 - `Country` → `languages`
 - `Country` → `flags`
 4. Anota en comentarios qué tipo de dato es cada campo (String, Map, List, etc.).
-

Ejercicio 2 — Modelo anidado básico (`Country` + `Name`)

Objetivo: crear un modelo que refleje la jerarquía de objetos.

Instrucciones:

1. Define dos clases: `Country` y `Name`.

```

class Name {
    final String common;
    final String official;

    Name({required this.common, required this.official});

    factory Name.fromJson(Map<String, dynamic> json) {
        return Name(
            common: json['common'] ?? '',
            official: json['official'] ?? '',
        );
    }
}

class Country {
    final Name name;
    final String region;
    final String capital;

    Country({
        required this.name,
        required this.region,
        required this.capital,
    });

    factory Country.fromJson(Map<String, dynamic> json) {
        return Country(
            name: Name.fromJson(json['name']),
            region: json['region'] ?? '',
            capital: (json['capital'] is List && json['capital'].isEmpty)
                ? json['capital'][0]
                : (json['capital'] ?? ''),
        );
    }
}

```

```
}
```

1. Crea una función `fetchCountry()` que:

- haga un GET a la API,
- parsee el JSON a un `Country`,
- y muestre `country.name.common` y `country.capital`.

Bonus: añade `try/catch` para errores de red o formato.

Ejercicio 3 — Modelos anidados y listas (`Country` + `Currency` + `Language`)

Objetivo: extender el modelo a estructuras dentro de *maps* y *lists*.

Instrucciones:

1. Añade nuevos modelos:

```
class Currency {
    final String name;
    final String symbol;

    Currency({required this.name, required this.symbol});

    factory Currency.fromJson(Map<String, dynamic> json) {
        return Currency(
            name: json['name'] ?? '',
            symbol: json['symbol'] ?? '',
        );
    }
}

class Language {
    final String code;
    final String name;
```

```

Language({required this.code, required this.name});

factory Language.fromJson(MapEntry<String, dynamic> entry) {
  return Language(code: entry.key, name: entry.value);
}
}

```

2. Modifica la clase `Country` :

```

class Country {
  final Name name;
  final String region;
  final String capital;
  final Map<String, Currency> currencies;
  final Map<String, String> languages;

  Country({
    required this.name,
    required this.region,
    required this.capital,
    required this.currencies,
    required this.languages,
  });

  factory Country.fromJson(Map<String, dynamic> json) {
    final currenciesJson = (json['currencies'] ?? {}) as Map<String, dynamic>;
    final languagesJson = (json['languages'] ?? {}) as Map<String, dynamic>;

    return Country(
      name: Name.fromJson(json['name']),
      region: json['region'] ?? '',

```

```

capital: (json['capital'] is List && json['capital'].isEmpty)
    ? json['capital'][0]
    : (json['capital'] ?? ''),
currencies: currenciesJson.map(
    (k, v) ⇒ MapEntry(k, Currency.fromJson(v)),
),
languages: languagesJson.map((k, v) ⇒ MapEntry(k, v.toString())),
);
}
}

```

3. Prueba en `main()` :

```

final country = await fetchCountryByName('peru');
print('País: ${country.name.common}');
print('Capital: ${country.capital}');
print('Moneda: ${country.currencies.values.first.name}');
print('Idioma principal: ${country.languages.values.first}');

```

Ejercicio 4 — Petición dinámica + manejo robusto de errores

Objetivo: ampliar el manejo de errores con `try/catch` y distintos tipos de excepción.

Instrucciones:

1. Implementa una función:

```

Future<Country?> fetchCountryByName(String name) async {
  try {
    final response = await http.get(Uri.parse('https://restcountries.com/v2/name/$name'));
    if (response.statusCode == 200) {
      final List data = jsonDecode(response.body);

```

```

    return Country.fromJson(data.first);
  } else if (response.statusCode == 404) {
    print('✗ País no encontrado: $name');
    return null;
  } else {
    throw Exception('Error del servidor: ${response.statusCode}');
  }
} catch (e) {
  print('⚠ Error de red o parseo: $e');
  return null;
}
}

```

1. Prueba con:

```

await fetchCountryByName('peru');
await fetchCountryByName('chile');
await fetchCountryByName('xyz'); // debe manejar el error

```

Aprendizaje:

- Diferenciar errores de red, de servidor y de formato.
- Controlar la estructura del JSON incluso cuando cambia.

Ejercicio 5 — Modelo con modelo dentro de modelo dentro de modelo

(Country → Name → NativeName → Official/Common)

Objetivo: profundizar en niveles anidados.

Instrucciones:

1. Añade submodelos:

```

class NativeName {
    final String official;
    final String common;

    NativeName({required this.official, required this.common});

    factory NativeName.fromJson(Map<String, dynamic> json) {
        return NativeName(
            official: json['official'] ?? '',
            common: json['common'] ?? '',
        );
    }
}

class Name {
    final String common;
    final String official;
    final Map<String, NativeName> nativeName;

    Name({
        required this.common,
        required this.official,
        required this.nativeName,
    });

    factory Name.fromJson(Map<String, dynamic> json) {
        final native = (json['nativeName'] ?? {}) as Map<String, dynamic>;
        return Name(
            common: json['common'] ?? '',
            official: json['official'] ?? '',
            nativeName: native.map(
                (k, v) => MapEntry(k, NativeName.fromJson(v)),
            ),
        );
    }
}

```

```
}
```

1. Usa esta `Name` dentro del modelo `Country`.
2. En `main()`, imprime algo así:

```
print('Nombre común: ${country.name.common}');  
print('Nombre oficial: ${country.name.official}');  
print('Nombre nativo español: ${country.name.nativeName['spa']?.common}');
```

Objetivo final: entender cómo modelar estructuras JSON jerárquicas y acceder a datos profundos.

Ejercicio 6 — Conversión inversa (`toJson()` en modelos anidados)

Objetivo: reconstruir el JSON desde los modelos.

Instrucciones:

1. Implementa `toJson()` en cada modelo (`Country`, `Name`, `NativeName`, `Currency`).
2. Genera un país desde la API y luego conviértelo de nuevo a JSON:

```
final country = await fetchCountryByName('peru');  
print(jsonEncode(country?.toJson()));
```

Aprendizaje:

- Cómo mantener consistencia de los modelos al serializar/deserializar.
- Ideal para integraciones con Firestore, APIs REST o almacenamiento local.

Ejercicio 7 — Búsqueda múltiple

Objetivo: usar `List<Country>` con nombres dinámicos.

Instrucciones:

1. Crea una función que reciba una lista de nombres (`["peru", "spain", "japan"]`).
2. Itera cada uno dentro de un `for` con `await fetchCountryByName(name)` .
3. Usa `try/catch` para que, aunque uno falle, el resto continúe.
4. Imprime resumen final de todos los países encontrados.